SCIENCE
ACADEMY

Naive Bayes: Weather Prediction Example

- **Problem**
  - Predict if kids play outside using past weather observations
- **Supervised learning problem**
  - Predictor vars:
    - `outlook = {sunny, overcast, rainy}`
    - `temperature = {hot, mild, cold}`
    - `humidity = {high, normal}`
    - `windy = {true, false}`
  - Response var:
    - `play = {yes, no}`

Outlook	Temp	Humidity	Windy	Play
Overcast	Cold	Normal	True	Yes
Overcast	Hot	High	False	Yes
Overcast	Hot	Normal	False	Yes
Overcast	Mild	High	True	Yes
Rainy	Cold	Normal	False	Yes
Rainy	Cold	Normal	True	No
Rainy	Mild	High	False	Yes
Rainy	Mild	High	True	No
Rainy	Mild	Normal	False	Yes
Sunny	Cold	Normal	False	Yes
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Sunny	Mild	High	False	No
Sunny	Mild	Normal	True	Yes

- **Training set**:
  - Samples for predictors and response from observations
  - Possible noise in data
    - E.g., kids have different preferences, some are sick illness, some have homework

Naive Bayes: Weather Prediction Example

- Use Bayes' rule to decide class H_j:

$$\Pr(H_j|\underline{\boldsymbol{E}}) = \frac{\Pr(\underline{\boldsymbol{E}}|H_j)\Pr(H_j)}{\Pr(\underline{\boldsymbol{E}})}$$

where:

- H_j: event to predict
  - E.g., `play = yes`
- $\underline{\boldsymbol{E}}$: event with feature values
  - E.g., `outlook=sunny, temperature=high, humidity=high, windy=yes`

Naive Bayes: Weather Prediction Example

- The **model** is:

$$\Pr(H_j|\underline{\boldsymbol{E}}) = \frac{\Pr(\underline{\boldsymbol{E}}|H_j)\Pr(H_j)}{\Pr(\underline{\boldsymbol{E}})}$$

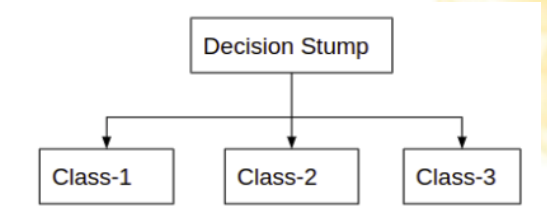
  where:
  - $\Pr(H_j)$: **prior probability** (probability of the outcome before evidence $\underline{\boldsymbol{E}}$)
    - E.g., $\Pr(\text{play} = \text{yes})$
    - Computed from training set as: $\Pr(H_j) = \sum_{k=1}^{N} \Pr(H_j \wedge \underline{\boldsymbol{E}}_k)$
  - $\Pr(\underline{\boldsymbol{E}})$: **probability of the evidence**
    - Computed from training set
    - Not needed as it is common to the probability of each class
  - $\Pr(\underline{\boldsymbol{E}}|H_j)$: **conditional probability**
    - Computed as independent probabilities (the "naive" assumption):

$$\begin{aligned}\Pr(\underline{\boldsymbol{E}}|H_j) &= \Pr(E_1 = e_1, E_2 = e_2, \ldots, E_n = e_n|H_j) \\ &\approx \Pr(E_1 = e_1|H_j) \cdot \ldots \cdot \Pr(E_n = e_n|H_j)\end{aligned}$$

- **Interpretation of Bayes theorem**
  - The prior is modulated through the conditional probability and the probability of the evidence

1-Rule

- Aka "tree stump", i.e., a decision tree with a single node
- **Algorithm**
  - Pick a single feature (e.g., `outlook`):
    - Most discriminant; or
    - Based on expert opinion
  - Choose the most frequent output for the feature's current value



- **Weather problem example**:
  - Pick `outlook` as single feature
  - Know predictor vars, e.g., `outlook = sunny`
  - Compute conditional probability using training set:

$$\Pr(\texttt{play = yes, no} \mid \texttt{outlook = sunny})$$

  - Output the predicted variable

Naive Bayes: Why Independence Assumption

- **Independence assumption** factors joint probability into marginal probabilities:

$$\begin{aligned}\Pr(\underline{\boldsymbol{E}}|H_j) &= \Pr(E_1 = e_1, E_2 = e_2, ..., E_n = e_n|H_j) \\ &\approx \Pr(E_1 = e_1|H_j) \cdot ... \cdot \Pr(E_n = e_n|H_j)\end{aligned}$$

- **Pros**:
  - Simplifies probability computation
  - Aids generalization due to few samples needed
- **Cons**:
  - Assumption may not be true

Estimating Probabilities: MLE

- **Maximum Likelihood Estimate** (MLE) estimates event probability by counting occurrences among all possible events:

$$\Pr(\underline{\boldsymbol{E}} = \underline{\boldsymbol{e}}') = \frac{\#I(\underline{\boldsymbol{E}} = \underline{\boldsymbol{e}}')}{K}$$

- For Naive Bayes, we need to estimate probability of each feature:

$$\Pr(E_i = e') = \frac{\#I(E_i = e')}{\sum_{j=1}^{V} \#I(E_i = e_k)}$$

- **Problem**
  - Value e' of feature E_i not in training set with output class H_j
  - Estimated probability $\Pr(E_i = e'|H_j) = 0$
  - Plugging $\Pr(E_i = e'|H_j) = 0$ into

$$\Pr(H_j|\underline{\boldsymbol{E}}) \approx \Pr(E_1 = e_1|H_j) \cdot \ldots \cdot \Pr(E_n = e_n|H_j)$$

  - Yields $\Pr(H_j|\underline{\boldsymbol{E}}) = 0 \implies$ impossible to decide output

Estimating Probabilities: Laplace Estimator

- Use Laplace estimator for events not in training set instead of MLE
- **Maximum likelihood estimate** (MLE)

$$\Pr(E_i = e') = \frac{\#I(E_i = e')}{\sum_{j=1}^{V} \#I(E_i = e_k)}$$

- **Laplace estimator**
  - Adds 1 to each count and V (number of feature values) to denominator:

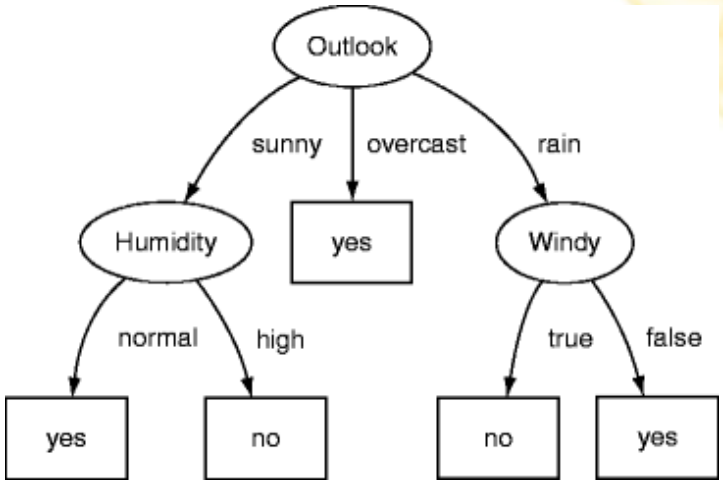
$$\Pr(E_i = e') = \frac{1 + \#I(E_i = e')}{\sum_{j=1}^{V}(1 + \#I(E_i = e'))} = \frac{1 + \#I(E_i = e')}{V + \sum_{j=1}^{V} \#I(E_i = e')}$$

  - Blends prior of equiprobable feature values with MLE estimates

- ***Decision Trees***

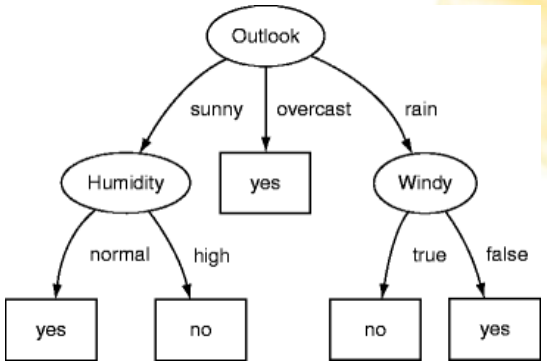
Decision Tree

- **Characteristics**
  - Supervised learning
  - Classification and regression
  - Non-parametric (i.e., no functional form)
- **Model**
  - Decision rules organized in a tree
- **Training**
  - Infer decision rules from data
  - Greedy divide-and-conquer
  - Worst case complexity: $O(2^n)$ with number of variables
- **Evaluation**
  - Evaluate model from root to leaves
  - Prediction cost: $O(log(n))$ with number of training points



Typical Decision Trees

- **Each node** tests a feature against a constant
  - Split input space with hyperplanes parallel to axes
- **Each feature**:
  - Tested once
  - Checks feature $x^{(j)}$ against threshold t
    - E.g., $x^{(j)}\{<, =, >\}t$
  - Result determines branch to follow
- **Leaves** represent decisions:
  - Class labels (e.g., classification)
    - Predict class or probability
  - Regression function



- Trees are non-linear models using variable interaction in `OR-AND` form:

$$y_i = (x_1 \geq x_1') \wedge (x_2 \geq x_2')...) \vee (...)$$

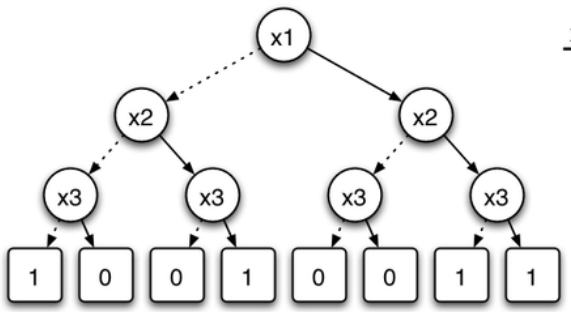
  - No re-converging paths: it's a tree!
  - Deeper tree $\implies$ more complex decision rules $\implies$ fitter model

Decision Trees: Pros

- Works for both **regression and classification**
- **Simple to understand and interpret**
  - White box model: explainable decisions
  - Visualizable
  - Can be created by hand
- Requires **little data preparation**
  - Invariant under feature scaling
    - No data normalization
    - No dummy variables
  - Handles numerical and categorical data
  - Robust to irrelevant features (implicit feature selection)
  - Missing values are treated:
    - As their own value; or
    - Assigned the most frequent value (i.e., imputation)
- **Scalable**
  - Performs well with large datasets

Decision Trees: Cons

- **Learning optimal decision tree** is NP-complete
  - Much worse than complexity of OLS
  - Algorithms use heuristics (e.g., greedy algorithms)
- **Risk of overfitting**
  - Solutions:
    - Pruning
    - Minimum samples at a leaf node
    - Max depth of trees
- **Some training sets are hard to learn**
  - E.g., `XORs`, parity

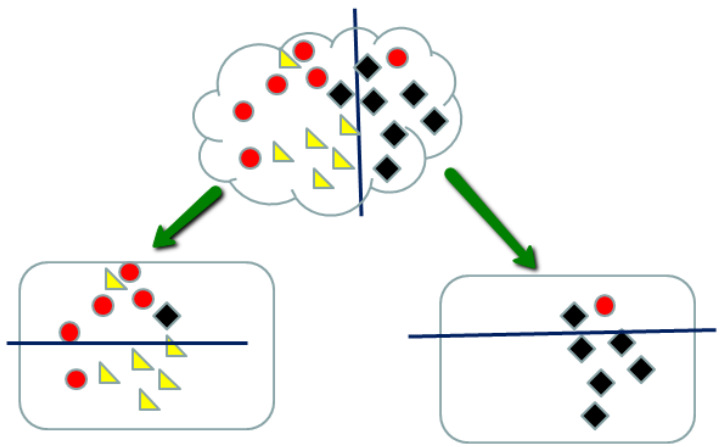


x1	x2	x3	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- **Unstable**
  - Small data variations greatly influence the tree
  - Solutions:
    - Ensemble learning / randomization

Learning Decision Trees: Intuition

- **Several algorithms**
  - ID3
  - C4.5 (proprietary)
  - CART (Classification And Regression Trees)
- Typically, the problem has a **recursive formulation**
  - Consider the current "leaf"
  - Find the variable/split ("node") that best separates outcomes into two groups ("leaves")
    - "Best" = samples with same labels grouped together
  - Continue splitting until:
    - Groups are small enough
    - Maximum depth is reached
    - Sufficiently "pure"



Splitting at One Node: Problem Formulation

- **Consider the *m*-th node** of a decision tree
  - Given $p_i = (\underline{\boldsymbol{x}}_i, y_i)$ where $i = 1, ..., N_m$, with training vectors $\underline{\boldsymbol{x}}_i$ and labels y_i
  - Candidate splits are $\theta = (j, t_m)$ with feature j and threshold t_m
  - Each split partitions data into subsets:

$$Q_L(j, t_m) = \{p_i = (\underline{\boldsymbol{x}}_i, y_i) : x_j \leq t_m\}$$
$$Q_R(j, t_m) = \{p_i \notin Q_L\}$$

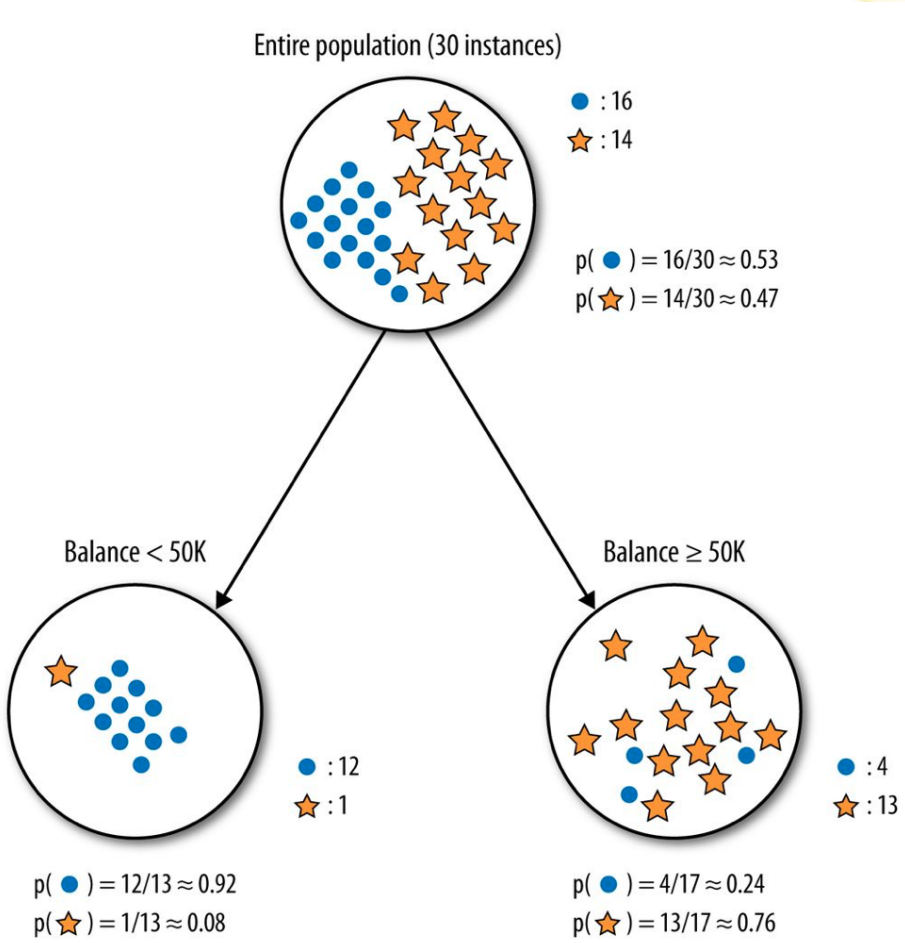
  - Impurity of a split is defined as:

$$H(j, t_m) = \frac{n_L}{N_m} H(Q_L) + \frac{n_R}{N_m} H(Q_R)$$

  - Find split $(j, t_m)^*$ minimizing $H(j, t_m)$
- **Recurse** on $Q_L(j, t_m)^*$ and $Q_R(j, t_m)^*$

Measures of Node Impurity for Classification

- **Measures of node impurity:**
  - Based on probabilities of choosing objects of different classes $k = 1, ..., K$ in the m-th node, i.e., p_k
  - Smaller impurity values are better
  - Less impurity means smaller probability of misclassification
- **Examples**
  1. Misclassification error
  2. Gini impurity index
  3. Information gain



Probability of Classification in a Node

- Assume m-th node has N_m objects x_i in K classes
- Compute class probability in m-th node as:

$$\hat{f}_{m,k} \triangleq \Pr(\text{pick object of class } k \text{ in } m\text{-th node }) = \frac{1}{N_m} \sum_{x_i \in \text{ node}} I(c(x_i) = k)$$

  where $y_i = c(x_i)$ is the correct class for element x_i

- E.g., if there are $N_m = 10$ objects belonging to $K = 3$ classes
  - 3 red, 6 blue, 1 green
  - The probability of classification $\hat{f}_{m,k}$ are:
    - Red $= 3/10$
    - Blue $= 6/10$
    - Green $= 1/10$

Misclassification Error: Definition

- You have several class probabilities p_i need a single probability
  - Consider worst case: most common class k' in node
- **Misclassification error**

$$H_M(p) \triangleq 1 - \max_k p_k$$

- **Binary classifier**
  - Best case (perfect purity)
    - Only one class in node
    - $H_M(p) = 0$
  - Worst case (perfect impurity)
    - 50-50 split between classes in node
    - $H_M(p) = 0.5$
- **Multi-class classifier** with K classes
  - Misclassification error has upper bound: $1/K$

Gini Impurity Index: Definition

- **Gini index** $H_G(p)$ is probability of picking an element randomly and classifying it incorrectly
  - By using the law of total probability:

$$H_G(p) = \Pr(\text{pick elem of } k\text{-th class}) \cdot \Pr(\text{misclassification} \mid \text{elem of } k \text{ class})$$

$$= \sum_{k=1}^{K} p_k \cdot (1 - p_k)$$

- **Binary classifier**
  - $H_G(p)$ is between 0 (perfect purity) and 0.5 (perfect impurity)
- **Multi-class classifier** with K classes
  - $H_G(p)$ has upper bound: $1 - K\frac{1}{K}^2 = 1 - \frac{1}{K}$

Information Gain: Definition

- Aka cross-entropy (remember entropy is $-p \log p$)
- **Information gain**

$$H_{IG} = -\sum_{k=1}^{K} p_k \cdot \log_2(p_k)$$

- **Binary classifier**
  - H_{IG} varies between 0 (perfect purity) and 1 (perfect impurity)
- **Multi-class classifier** with K classes
  - H_{IG} has upper bound: $\log K$

Measures of Impurity: Examples

- Consider the case of 16 elements in a node , belonging to 2 classes
- If all elements are of the same class:
  - Misclassification error $= 0$
  - Gini index $= 1 - (1 - 0) = 0$
  - Information gain $= -(1 \cdot \log_2(1) - 0 \cdot \log_2(0)) = 0$
- If one element is of one class:
  - Misclassification error $= 1 - \max(\frac{1}{16}, \frac{1}{15}) = \frac{1}{16}$
  - Gini index $= 1 - ((\frac{1}{16})^2 + (\frac{1}{15})^2) = 0.12$
  - Information gain $= -(\frac{1}{16}\log_2(\frac{1}{16}) + \frac{15}{16}\log_2(\frac{15}{16})) = 0.34$
- If elements are split in the two classes equally:
  - Misclassification error $= \frac{8}{16} = 0.5$
  - Gini index $= 1 - (\frac{8}{16}^2 + \frac{8}{16}^2) = 0.5$
  - Information gain $= -(\frac{8}{16}\log_2(\frac{8}{16}) + \frac{8}{16}\log_2(\frac{8}{16})) = 1$